

PROP SEGONA ENTREGA

Versió 2.1

Grup 13.2 - Alicia Ageno

BARANDA DOMINGUEZ, JORDI - jordi.baranda@estudiantat.upc.edu

CLUSELLAS CÁNOVAS, JUAN - juan.clusellas@estudiantat.upc.edu

SOLANES SERRAT, FERRAN - ferran.solanes@estudiantat.upc.edu

TURU ROCA, MARC - marc.turu@estudiantat.upc.edu



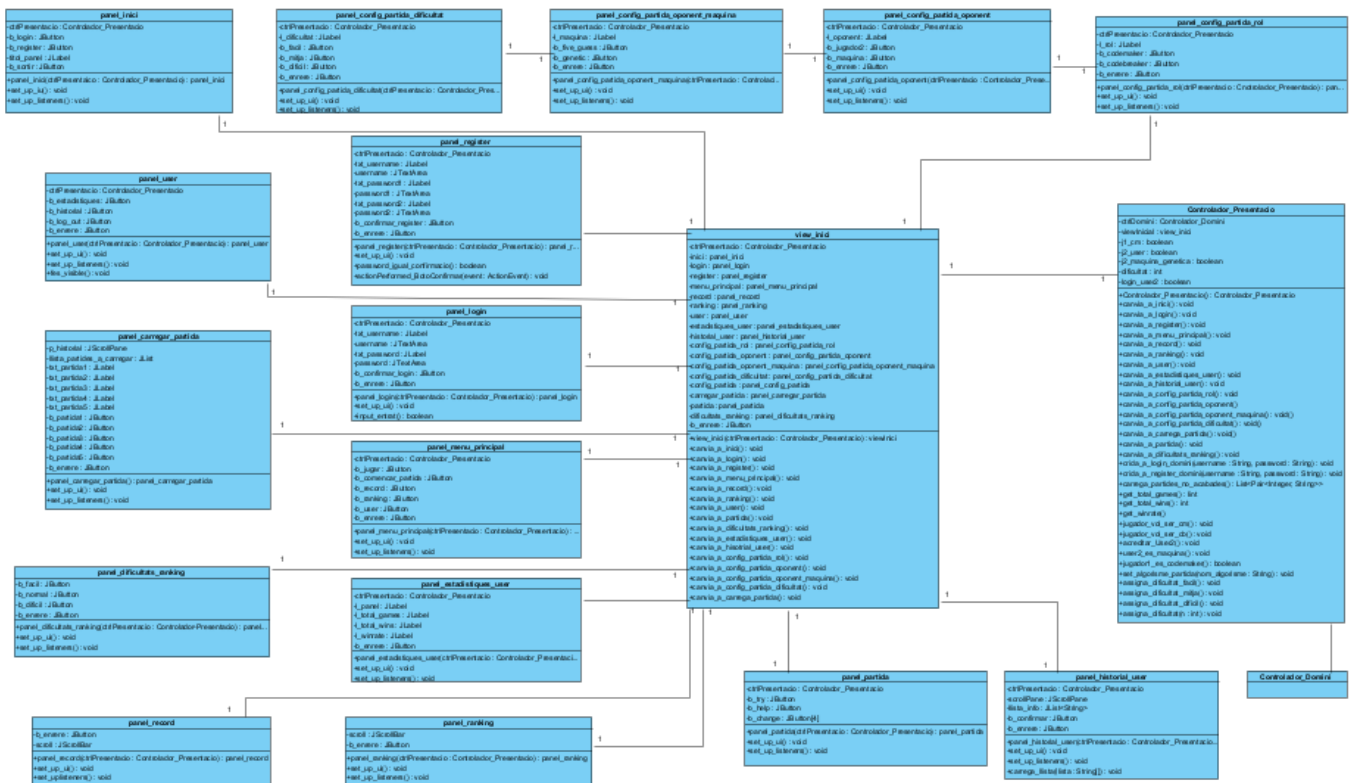
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



1. Capa de presentació	4
1.1 Diagrama de la capa de presentació	4
1.2 Descripció de les classes	5
1.2.1 panel_inici	5
1.2.2 label_register	5
1.2.3 panel_login	6
1.2.4 panel_menu_principal	6
1.2.5 panel_user	7
1.2.6 panel_estadistiques_user	7
1.2.7 panel_historial_user	8
1.2.8 panel_config_partida_rol	8
1.2.9 panel_config_partida_oponent	9
1.2.10 panel_config_partida_oponent_maquina	9
1.2.11 panel_config_partida_dificultat	10
1.2.12 panel_carregar_partida	10
1.2.13 panel_partida	11
1.2.14 panel_dificultats_ranking	11
1.2.15 panel_ranking	11
1.2.16 panel_record	12
1.2.17 view_inici	12
2. Capa de domini	14
2.1 Diagrama de la capa de domini	14
2.2 Descripció de les classes	15
2.2.1 User	15
2.2.2 User_persona	16
2.2.3 User_maquina	17
2.2.4 Five_guess_algorithm	18
2.2.5 Genetic_algorithm	19
2.2.6 Cromosoma	19
2.2.7 Partida	20
2.2.8 Ronda	21
2.2.9 Seqüència	22
2.2.10 Seqüència_intentada	23
2.2.11 Seqüència_verificació	23
2.2.12 Rànk	24
2.2.13 Record	24
2.2.14 RecordDouble	25
2.2.15 RecordInteger	25
2.2.16 RecordLong	26
2.2.17 Type_user	26
2.2.18 dificultats	26
2.2.19 colors	27
2.2.20 Palr	27

3. Capa de persistencia	28
3.1 Diagrama de la capa de persistència	28
3.2 Descripció de les classes	28
3.2.1 ctrl_list_user	29
3.2.2 ctrl_user	29
3.2.3 ctrl_partida	30
3.2.4 ctrl_ranking	30
3.2.5 ctrl_record	31
4. Estructura de dades i algoritmes	32
4.1 Estructures de dades	32
4.2 Algoritmes	33
Five Guess Algorithm	33
List	34
Arrays	34
ArrayList	34
Math.pow()	34
Random	34
generateCodes()	34
getScore()	35
getColors()	35
filterCodes()	35
getBestGuess()	35
Genetic Algorithm	37
Comparació de la performance d'algoritmes	39



1.2 Descripció de les classes

Diferents panells:

Tots els panells comparteixen dues funcions privades comunes. Una que es diu *set_up_ui()* que distribueix els components per el panell, i una altra que es diu *set_up_listeners()* que serveix per a assignar els listeners a cada contenidor.

1.2.1 panel_inici

Aquest panell serà el primer que ens aparegui quan iniciem l'aplicació. Ens mostra el títol i pregunta si es vol fer una acreditació d'usuari(*login*) o si es vol registrar un nou usuari(*register*). En quant a components, tindrem un *JLabel* per mostrar el títol de l'aplicació i tres *JButton* que, depenent del que apremem, ens portarà a fer l'acreditació o el registre de l'usuari o ens permetrà tancar l'aplicació.

Atributs:

- *JButton* b_login: botó per passar a acreditar un usuari.
- *JButton* b_register: botó per passar a registrar a un usuari.
- *JLabel* titol_app: títol de l'aplicació.
- *JButton* b_sortir: botó per tancar l'aplicació.

Mètodes:

- *panel_inici()*: constructora del panell amb els components inicials.

1.2.2 label_register

Aquest label serà el que veurem un cop haguem apretat el botó de register. Aquí es farà tot el demanament de dades a l'usuari per a registrar un nou usuari. En el nostre cas es demana el nom d'usuari que es vol tindre i la contrasenya, dues vegades per confirmar que no s'han fet errors d'escriptura al entrar-la. Aquesta vista té; un *JButton* per anar enrere(a la pantalla inicial), un *JButton* per confirmar que s'han entrat els paràmetres i es vol fer la comprovació d'usuari, i 3 *JTextArea* on s'entra el nom_usuari, la contrasenya i la confirmació de contrasenya en el respectiu ordre. Cada area de text té un *JLabel* al costat indicant el camp que s'ha d'entrar. En cas que l'usuari que es vol registrar ja existeixi (mateix nom), no es podrà registrar, i s'haurà de acreditar en cas que l'usuari sigui seu, o canviar el nom d'usuari a fer servir.

Atributs:

- *JLabel* txt_username: indicador del camp *username*
- *JTextArea* username: camp on s'entra el *username*
- *JLabel* txt_password1: indicador del camp *password*

- `TextArea password1`: camp on s'entra la *password*
- `JLabel txt_password2`: indicador del camp per confirmar *password*
- `TextArea password2`: camp on s'entra la confirmació *password*
- `JButton b_confirmar_register`: botó per seguir endavant amb el registre.
- `JButton b_enrere`: botó per sortir del panell actual(tancar aplicació).

Mètodes:

- `panel_register()`: Creadora del panell per registrar un usuari.

1.2.3 panel_login

Aquest panell és per quan un usuari ja prèviament registrat vol entrar i poder veure la seva configuració, les seves estadístiques, les seves partides, etc. Aquest panell és similar al de registre, però sense el camp de confirmació de contrasenya, per tant 2 *JLabel* i 2 *TextArea* per entrar el *username* i *password*. També hi ha un botó per tirar enrere i retornar a la pantalla d'inici. En cas que l'usuari no existeixi, s'ha de notificar a l'usuari que no existeix aquell usuari. L'usuari ha de decidir si vol registrar-se o revisar que hagi entrat bé les credencials.

Atributs:

- `JLabel txt_username`: indicador del camp *username*
- `TextArea username`: camp on s'entra el *username*
- `JLabel txt_password`: indicador del camp *password*
- `TextArea password`: camp on s'entra la *password*
- `JButton b_confirmar_login`: botó per seguir endavant amb l'acreditació
- `JButton b_enrere`: botó per anar a la pantalla anterior.

Mètodes:

- `panel_login()`: creadora del panell per acreditar un usuari.

1.2.4 panel_menu_principal

Aquest panell s'usa quan hem fet el login/registre del nostre usuari. Aquí se'ns proposen 5 possibilitats: començar una nova partida, carregar una partida ja començada, mirar els rankings, mirar els records o bé accedir al menú d'usuari. Per a cada una de les possibilitats, tindrem un botó, implementat en *JButton* que ens canviï el *contentPane* al panell corresponent. També tindrem un botó per anar enrere a la pantalla anterior. En aquest panell no poden saltar errors/excepcions ja que les accions estan molt limitades.

Atributs:

- `JButton b_jugar`: botó per indicar que volem començar una partida.
- `JButton b_carregar_partida`: botó per indicar que volem carregar una partida.

- JButton b_records: botó per indicar que volem veure els diferents records.
- JButton b_rankings: botó per indicar que volem veure un dels diferents rankings.
- JButton b_user: botó per indicar que volem entrar al menú de l'usuari
- JButton b_enrere: botó per retornar al inici(login/register).

Mètodes:

- panel_menu_principal(): creadora del panell del menú principal amb les opcions que pot fer l'usuari

1.2.5 panel_user

En aquest panell podem trobar el menú amb les opcions que podem fer implicant l'usuari amb el que ens hem acreditat. En aquest menú trobem una opció per veure les estadístiques del perfil(partides jugades, partides guanyades, winrate), una opció per veure l'historial de les partides que hem anat fent (**només les acabades**) i una opció per fer "log out" i tornar a posar el panell principal. Totes aquestes funcionalitats es poden executar apretant un botó per a cada una. Com en casos anteriors, també tenim un botó per a tornar a la pantalla anterior(menú principal). Els botons estan implementats amb *JButtons*.

Atributs:

- JButton b_estadistiques: botó per veure les estadístiques de l'usuari
- JButton b_historial: botó per a veure l'historial de partides de l'usuari
- JButton b_logout: botó per a sortir del compte de l'usuari actual
- JButton b_enrere: botó per tornar al menú principal

Mètodes:

- panel_user(): Constructora del panell del menú de l'usuari.

1.2.6 panel_estadistiques_user

En aquesta vista podem veure les estadístiques de l'usuari. Aquestes són: partides totals, partides guanyades, winrate. Es mostren en *JLabel* ja que no són editables per l'usuari.

Atributs:

- JLabel total_games: contador de les partides totals de l'usuari.
- JLabel total_wins: contador de les partides guanyades.
- JLabel winrate: ratio entre partides guanyades i partides jugades.
- JButton b_enrere: botó per anar a la pantalla prèvia (menú principal).

Mètodes:

- panel_estadistiques_user(): Creadora del panell per veure les estadístiques de l'usuari.

1.2.7 panel_historial_user

En aquest panell es mostra l'historial de partides acabades de l'usuari per a que ell pugui carregar-les i veure quins guess va provar juntament amb les correccions que es van fer. En aquest panell tindrem el botó de tirar enrere, com els altres panells i també tindrem un JScrollPane, amb una List a dins d'aquest, per a mostrar la informació de cada partida(ronda per la que es va, la dificultat, el rol i la data en que es va començar la partida). També tindrem un botó per a que l'usuari pugui avisar a l'aplicació que vol veure la partida seleccionada. Per tant l'usuari podrà triar, amb més info que només l'identificador de la partida, quina partida vol veure. Per accedir a la partida cal seleccionar la partida que volem i apretar el botó de confirmar.

Atributs:

- JScrollPane scrollPane: panell que conté la llista d'informació
- JList llista_info: llista amb la informació de les partides antigues
- JButton b_confirmar: botó per confirmar que volem la partida seleccionada.
- JButton b_enrere: botó per anar a la pantalla prèvia (menú principal).

Mètodes:

- panel_historial_user(): Creadora del panell per a veure l'historial del user

1.2.8 panel_config_partida_rol

En aquesta vista se'ns presenten els diferents paràmetres per a configurar la partida: jugar vs màquina/jugador, rol que vol jugar l'usuari principal de la partida i la dificultat de la partida. Tots tres paràmetres es presentaran de la mateixa manera (ho fem amb botons). L'usuari té unes opcions definides per els desenvolupadors del joc, per tant no pot haver-hi errors com ara dificultat que no existeix, rol que no existeix o tipus de partida que no existeix. Com en els anteriors casos, tindrem un botó per tirar enrere a la anterior pantalla.

Atributs:

- JLabel l_rol: Label per indicar el que s'ha de decidir el rol
- JButton b_codemaker: botó per seleccionar ser codemaker
- JButton b_codebreaker: botó per seleccionar ser codebreaker
- JButton b_enrere: botó per anar a la pantalla anterior.

Mètodes:

- panel_config_partida_rol(): Creadora del panell de configuració de la partida per triar el rol.

1.2.9 panel_config_partida_oponent

En aquest panell es presenten les opcions contra les que jugar la partida. S'indica amb un JLabel que s'està triant l'oponent contra el que es jugarà. Es dóna l'opció de jugar contra un altre usuari(en aquest cas el segon usuari s'haurà de registrar/loguejar) o contra una màquina(en aquest cas s'haurà de decidir quina màquina es vol usar). Com en altres pantalles, hi ha un botó per a tirar enrere a la pantalla anterior.

Atributs:

- JLabel l_oponent: Label per indicar el que s'ha de decidir l'oponent.
- JButton b_jugador2: botó per seleccionar un altre jugador com a oponent.
- JButton b_maquina: botó per seleccionar una màquina com a oponent.
- JButton b_enrere: botó per anar a la pantalla anterior(panel_config_partida_rol).

Mètodes:

- panel_config_partida_oponent(): Creadora del panell de configuració de la partida per triar l'oponent.

1.2.10 panel_config_partida_oponent_maquina

En aquest panell es presenten les màquines disponibles contra les que jugar. A aquest panell s'hi entra si, i només si, s'ha decidit jugar com a codemaker i contra una màquina. En aquesta pantalla trobarem un JLabel indicant que estem seleccionant una màquina per a que intenti trencar el nostre codi. Es pot seleccionar entre Five-Guess i Genètic a través de dos JButton, un per a cada màquina. Com en altres pantalles, tenim un JButton per a anar a la pantalla anterior.

Atributs:

- JLabel l_maquina: Label per indicar el que s'ha de decidir la màquina contra la que jugar.
- JButton b_five_guess: botó per seleccionar five_guess com a oponent.
- JButton b_genetic: botó per seleccionar genetic com a oponent.
- JButton b_enrere: botó per anar a la pantalla anterior(panel_config_partida_oponent).

Mètodes:

- panel_config_partida_oponent_maquina(): Creadora del panell de configuració de la partida per triar la maquina contra la que jugar.

1.2.11 panel_config_partida_dificultat

En aquesta pantalla es presenten les diferents dificultats amb les que es pot jugar la partida. Tenim un JLabel per indicar que estem triant la dificultat de la partida. A part d'això tenim 3 JButton, un per cada dificultat i un JButton per anar a l'anterior pantalla (panel_config_partida_oponent_maquina).

Atributs:

- JLabel l_dificultat: Label per indicar el que s'ha de decidir la dificultat a la que jugar.
- JButton b_facil: botó per seleccionar dificultat facil.
- JButton b_normal: botó per seleccionar dificultat normal.
- JButton b_dificil: botó per seleccionar dificultat dificil.
- JButton b_enrere: botó per anar a la pantalla anterior (panel_config_partida_oponent_maquina).

Mètodes:

- panel_config_partida_dificultat(): Creadora del panell de configuració de la partida per triar la dificultat a la que jugar.

1.2.12 panel_carregar_partida

En aquesta vista podem veure un cert nombre de partides que podem carregar per a seguir jugant. Això se'ns presenta amb un JScrollPane(per a poder moure'ns per la llista), on es mostra la següent informació per a cada partida(última ronda jugada, dificultat, rol i la data en que es va crear la partida) dins d'una JList continguda al JScrollPane. Com sempre tindrem un botó per anar a la pantalla anterior. A aquesta vista arribem desde el menú principal.

Atributs:

- JScrollPane p_historial: Panell que contindrà la llista amb les diferents partides a carregar.
- JList llista_partides_a_carregar: llista de Strings amb la informació de les diferents partides a carregar.
- JButton b_enrere: botó per anar a la pantalla prèvia (menú principal).

Mètodes:

- panel_carregar_partida(): Creadora del panell per a carregar una partida nova.

1.2.13 panel_partida

En aquest panell és on es podrà jugar la partida. El codebreaker podrà inserir la seva seqüència intentada a través dels botons que se li proporcionaran per canviar els colors i, una vegada apreti el botó de try, el codemaker podrà fer la correcció de la seqüència.

A més, disposa d'un botó de help per demanar ajuda i un botó per anar a la pantalla anterior.

Atributs:

- JButton b_try: botó per provar la seqüència entrada
- JButton b_help: botó per demanar ajuda
- JButton [4] b_change: botó per canviar el color a entrar a la seqüència
- JButton b_enrere: botó per anar a la pantalla prèvia (menú principal).

Mètodes:

- panel_partida(): Creadora del panell per a jugar una partida.

1.2.14 panel_dificultats_ranking

En aquest panell es mostren les 4 opcions per a veure els rankings: facil, normal, dificil, pvp. Al seleccionar un es canvia a veure el ranking corresponent. Per a fer aquesta interfície podem fer servir 4 botons, un per cada tipus de ranking, més un botó per a tirar enrere.

Atributs:

- JButton b_facil: botó per anar al ranking fàcil
- JButton b_normal: botó per anar al ranking normal
- JButton b_dificil: botó per anar al ranking difícil
- JButton b_enrere: botó per anar a la pantalla prèvia (menú principal).

Mètodes:

- panel_dificultats_ranking(): Creadora del panell per decidir de quina dificultat volem veure el ranking.

1.2.15 panel_ranking

En aquest panell es mostrarà el ranking (segons la dificultat) del joc, on es mostrarn tots els jugadors (que es podran veure complets fent un scroll) i es podrà anar a la pantalla anterior.

Atributs:

- JScrollBar scroll: servirà per veure tot el rànkig que forma el joc fent "scroll"

- JButton b_enrere: botó per anar a la pantalla prèvia (dificultats dels ranking).

Mètodes:

- panel_ranking(): Creadora del panell per a veure el ranking de la dificultat decidida.

1.2.16 panel_record

En aquesta pantalla es mostaran els diferents recors que hi han en el joc. El records s'imprimiran en forma de string on passar el nom del record, els punts, la modalitat i qui ha fet el record. Hi haurà una barra de navegació en cas de que no hi capiguin en la pantalla.

Atributs:

- JButton b_enrere: botó per anar a la pantalla prèvia (menú principal).
- List<Record>: Llista on estan tots el records.

Mètodes:

- panel_llista_records(): Creadora del panell per veure els diferents records.

Vistes:

1.2.17 view_inici

Aquesta vista és la principal i la única dins de la nostre capa de presentació.

Consisteix en un JFrame, que va canviant de diferents panells, com pot ser el de jugar partida o el de veure records.

Atributs:

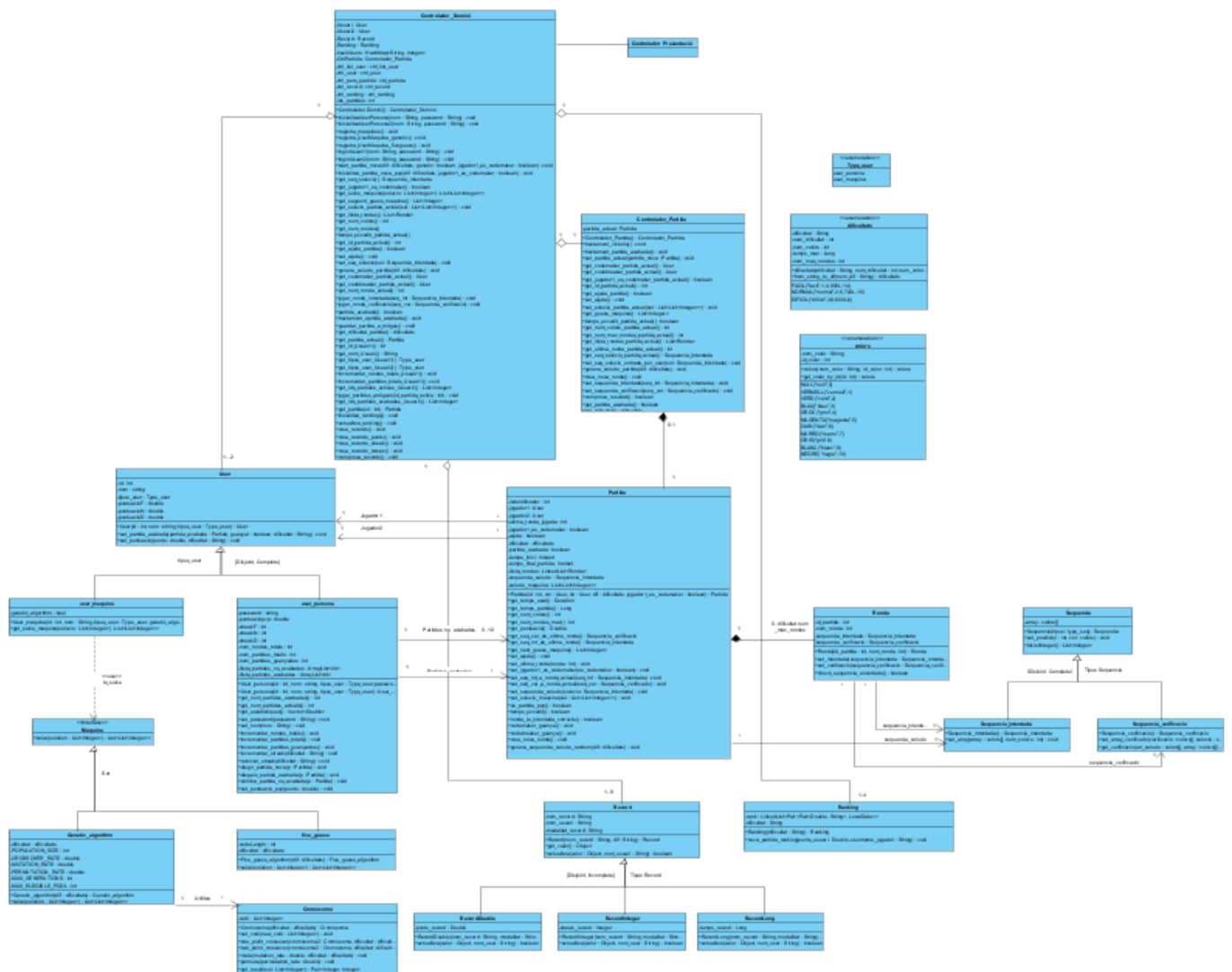
- panel_inici: Crida al panell de inici.
- panel_login: Crida al panell de login.
- panel_register: Crida al panell de register.
- panel_menu_principal: Crida al panell de menu principal.
- panel_record: Crida al panell de record.
- panel_ranking: Crida al panell de ranking.
- panel_user: Crida al panell de user.
- panel_estadistiques_user: Crida al panell de estadistiques user.
- panel_historial_user: Crida al panell de historial user
- panel_config_partida_rol: Crida al panell de configuracio del rol de partida.
- panel_config_partida_oponen: Crida al panell de configuracio del oponent.

- panel_config_partida_oponent_maquina: Crida al panell de configuracio del oponent maquina.
- panel_config_partida_dificultat: Crida al panell de configuracio de dificultat.
- panel_carregar_partida: Crida al panell de carregar partida.
- panel_partida: Crida al panell de partida.
- panel_dificultats_ranking: Crida al panell de dificultats de ranking.

Mètodes:

El següent llistat de mètodes, el que fa és canviar el Jpanel del JFrame a un altre, i cada mètode o fa a un diferent, per exemple canvia_a_login(), canvia al panell de login.

- public void canvia_a_inici()
- public void canvia_a_login()
- public void canvia_a_register()
- public void canvia_a_menu_principal()
- public void canvia_a_record()
- public void canvia_a_ranking()
- public void canvia_a_user()
- public void canvia_a_partida()
- public void canvia_a_dificultats_ranking()
- public void canvia_a_estadistiques_user()
- public void canvia_a_historial_user()
- public void canvia_a_config_partida_rol()
- public void canvia_a_config_partida_oponent()
- public void canvia_a_config_partida_oponent_maquina()
- public void canvia_a_config_partida_dificultat()
- public void canvia_a_carregar_partida()



2.2 Descripció de les classes

Aquesta descripció de les classes es farà sobre totes les que apareixen al diagrama UML i es comentaran tots els enumerations usats i que presents en el mateix.

2.2.1 User

Aquesta classe fa referència als usuaris que jugaran al joc. Aquest usuaris poden ser de tipus persona o de tipus màquina (`user_persona` i `user_maquina`). Els users tenen un id únic que els identifica. A part, té altres atributs com `nom`, `tipus_user`, i altres que serviran per desenvolupar diferents funcionalitats del mateix, com les puntuacions d'aquest.

En aquesta superclasse estan implementats la majoria de getters i setters que permeten obtenir els valors d'aquests atributs mencionats, tot i que la majoria d'operacions és desenvolupen a la subclasse `user_persona`.

Atributs

- **protected int id:** Identificador de l'usuari
- **protected String nom:** Nom amb el que es registrarà l'usuari i amb el qual apareixerà a l'interfície del joc
- **protected Type_user tipus_user:** Enumeration que diferencia els dos tipus d'usuari
- **protected double puntuacioF:** Puntuació de l'usuari de només les partides que ha jugat de tipus fàcil.
- **protected double puntuacioN:** Puntuació de l'usuari de només les partides que ha jugat de tipus normal.
- **protected double puntuacioD:** Puntuació de l'usuari de només les partides que ha jugat de tipus difícil.

Mètodes principals

- **public void set_partida_acabada(Partida partida_acabada, boolean gunyat, String dificultat):** Funció que serveix per a realitzar modificacions sobre l'usuari i els seus atributs després de la finalització d'una partida, com: actualitzar la puntuació de l'usuari, actualitzar les llistes de partides acabades (+1) i no acabades (-1), comprovar si cal actualitzar el streak (ratxa) amb +1 o a 0 (tot això depenent de si la partida és pvp o no i de si l'usuari és `user_persona` o no).
Algunes d'aquestes modificacions són simplement per a actualitzar atributs d'una forma simple i les altres per cridar a altre funcions com `set_puntuacio()`.

- **public void set_puntuacio(int punts_base, int win_bonus, int punts_penlaitzacio_rondes, String dificultat):** Aquesta funció, cridada quan s'acaba una partida i s'han de restablir les puntuacions, actualitza els valors de les puntuacions segons el tipus de dificultat que li arriba com a paràmetre perquè aquest, posteriorment, pugui ser tingut en compte per futures actualitzacions de rànkings i possibles rècords.

2.2.2 User_persona

Aquesta subclasse de User, ens permet implementar funcionalitats extres de User que només poden ser usades/provades per aquest tipus d'usuari.

Bàsicament ens serveix per diferenciar el tipus de partides que juga cada user i quins atributs el componen. Aquest user_persona podrà jugar partides PvsP i obtenir puntuacions d'aquest tipus, a la vegada que tenir unes certes estadístiques de les partides jugades.

A part, aquest tipus d'usuari podrà tenir contrasenya per a poder fer el registre i el log in per a identificar-se .

Atributs

- **Tots els heredats de la superclasse User**
- **private String password:** Contrassenya que utilitzarà l'usuari per registrar-se i, posteriorment, fer el seu log in
- **private double puntuacioPvsP:** Puntuació de l'usuari de només les partides que ha jugat de tipus PvsP
- **private int streakF:** Ratxa de partides fàcils guanyades que té l'usuari
- **private int streakN:** Ratxa de partides normals guanyades que té l'usuari
- **private int streakD:** Ratxa de partides difícils guanyades que té l'usuari
- **protected int num_rondes_totals:** Número de rondes totals que ha jugat l'usuari des de que està registrat
- **protected int num_partides_totals:** Número de partides totals que ha jugat l'usuari al joc
- **protected int partides_guanyades:** Número de partides que ha guanyat l'usuari dins de totes les que ha jugat.
- **protected List<Partida> llista_partides_no_acabades:** Array llista de tipus Partida que conté totes les partides que l'usuari no ha acabat (no ha ni guanyat ni perdut)
- **protected List<Partida> llista_partides_acabades:** Array llista de tipus Partida que conté totes les partides que l'usuari ha acabat (ha guanyat o perdut)

Mètodes principals

- **public vector<Double> get_estadistiques():** Aquesta funció permet retornar i mostrar les estadístiques d'un usuari de tipus persona. D'aquesta manera, i retornant-les amb un vector, es pot accedir d'una manera senzilla als valors dels atributs del usuari que poden ser útils per fer comparacions entre ells o per, simplement, mostrar-los per pantalla.
- **public void incrementar_streak(String dificultat):** Aquesta funció serveix per augmentar la ratxa del tipus de partida que se li passa per paràmetre i de la qual, per tant, l'usuari ha guanyat i podrà mostrar per pantalla
- **public void reiniciar_streak(String dificultat):** Aquesta funció serveix per reiniciar la ratxa del tipus de partida que se li passa per paràmetre i de la qual, per tant, l'usuari ha perdut
- **public void set_puntuacio_PvsP(int punts_base, int win_bonus, int punts_penalitzacio_rondes):** Igual que a la superclasse però amb la puntuació de tipus PvsP

2.2.3 User_maquina

Aquesta subclasse de User, ens permet implementar funcionalitats extres de User que només poden ser usades/provades per aquest tipus d'usuari.

Bàsicament ens serveix per saber quin tipus d'algorisme s'utilitzarà per a la resolució del joc i, en conseqüència, quin tipus de màquina és.

En aquesta classe, la màquina, depenent de quin tipus d'algorisme usi, serà la qual jugarà les partides, resolent les seqüències a endevinar.

Atributs

- **Tots els heredats de la superclasse User**
- **private boolean genetic_algorithm:** Booleà que serveix per diferènciar quin tipus d'algorisme es farà servir i quin tipus de màquina és l'usuari2

Mètodes principals

- **List<List<Integer>> get_solve_maquina(List<Integer> solucio):** És, bàsicament, la funció que crida a l'algoritme per resoldre el joc. Encara que sigui només una

funció que crida a una altra, l'algoritme consta de tanta complexitat, que val la pena mencionar-la.

Aquesta interfície màquina, implementa els dos algoritmes per resoldre la seqüència creada pel codemaker.

2.2.4 Five_guess_algorithm

Aquesta classe representa l'algorisme five_guess i és la que s'encarrega de la implementació d'aquest i la resolució de la seqüència utilitzant-lo.

Atributs

- **private int codeLength:** Té el valor 4 per defecte per simbolitzar la llargada del codi a provar
- **private dificultat dificultat:** Dificultat de la partida en la qual haurà de ser aplicat l'algorisme de resolució

Mètodes principals

- **public List<List<Integer>> solve(List<Integer> solution):** Funció principal que permet resoldre la seqüència proposada
- **private List<List<Integer>> generateCodes(List<Integer> colors, int solutionLength):** Funció que genera tots els codis possibles a partir d'una llista de colors i una llargada de la solució possible
- **private List<List<Integer>> filterCodes(List<List<Integer>> possibleCodes, List<Integer> guess, List<Integer> result):** Funció que filtra tots els codis possibles que tenen el mateix resultat que la intentada
- **private List<Integer> getBestGuess(List<List<Integer>> possibleCodes):** Funció que retorna la millor solució possible donada una llista de possibles solucions
- **private Integer[] getScore(List<Integer> guess, List<Integer> secretCode):** Funció que retorna el resultat (blanques i negres) d'una possible solució
- **private Integer[] getColors():** Funció que retorna una llista de colors depenent de la dificultat de la partida

2.2.5 Genetic_algorithm

Aquesta classe representa l'algorisme genètic i és la que s'encarrega de la implementació d'aquest i la resolució de la seqüència utilitzant-lo.

Atributs

- **private static final int POPULATION_SIZE:** Té valor 150 per defecte
- **private static final double CROSSOVER_RATE:** Té valor 0.5 per defecte
- **private static final double MUTATION_RATE:** Té valor 0.03 per defecte
- **private static final double PERMUTATION_RATE:** Té valor 0.03 per defecte
- **private static final int MAX_GENERATIONS:** Té valor 1000 per defecte
- **private static final int MAX_ELEGIBLE_POOL:** Té valor 300 per defecte

Mètodes principals

- **private List<Cromosoma> init_poblacio():** Funció que serveix per inicialitzar la població de la generació i que garanteix que no hi hauran cromosomes repetits
- **private double fitness(Cromosoma c, List<Cromosoma> cromosomes_previs, List<Integer> codi_solucio):** Funció que implementa el càlcul del fitness d'un cromosoma segons l'article:
<https://lirias.kuleuven.be/bitstream/123456789/184247/2/Mastermind>
- **private List<Cromosoma> genera_nova_poblacio(List<Integer> codi_solucio, List<Cromosoma> codis_previs, List<Cromosoma> poblacio_anterior):** Funció per a generar la nova població, fent servir el mètode de la ruleta, segons descrit a:
<https://lirias.kuleuven.be/bitstream/123456789/184247/2/Mastermind>
- **public List<List<Integer>> solve(List<Integer> solution):** Funció principal per a resoldre la seqüència proposada

2.2.6 Cromosoma

Aquesta classe representa el Cromosoma que serà utilitzat (varies vegades) en la resolució de la seqüència quan es dugui a terme per part de l'algorisme genetic.

Atributs

- **private List<Integer> codi = new ArrayList<>(4)**

Mètodes principals

- **public Cromosoma one_point_crossover(Cromosoma cromosoma2, dificultats dificultat):** Funció per a fer el crossover de un sol punt entre dos cromosomes
- **public Cromosoma two_point_crossover(Cromosoma cromosoma2, dificultats dificultat):** Funció per a fer el crossover de dos punts entre dos cromosomes
- **public void muta(double mutation_rate, dificultats dificultat):** Funció per a fer la mutació d'un cromosoma
- **public void permuta(double permutation_rate):** Funció per a fer la permutació d'un cromosoma
- **public Pair<Integer, Integer> get_result(List<Integer> sol):** Funció per a avaluar el fitness d'un cromosoma, compara el cromosoma amb un altre com si aquell fos la solució

2.2.7 Partida

Aquesta classe serveix per identificar totes les característiques i particularitats que formen una partida.

Per a fer això, s'utilitzen atributs que formen i singularitzen cada partida en concret.

Cada partida utilitza una identifiador únic que l'identifica.

Atributs

- **private int identificador:** Identificador únic de la partida
- **private User jugador1:** Primer usuari que jugarà aquella partida
- **private User jugador2:** Segon usuari que jugarà aquella partida
- **private int ultima_ronda_jugada:** Número de la última ronda jugada en aquella partida
- **private boolean jugador1_es_codemaker:** Booleà que permet identificar si el jugador1 és el codemaker de la partida
- **private boolean ajuda:** Booleà que indica si s'ha fet servir la ajuda o no en aquesta partida
- **private dificultats dificultat:** Tipus de dificultat de la qual és aquella partida. És constant durant tota la partida
- **private boolean partida_acabada:** Booleà que indica si aquella partida està acabada o no
- **private Instant temps_inici:** Moment en el qual s'ha començat la partida

- **private Instant temps_final_partida:** Moment en el qual s'ha acabat la partida
- **private List<Ronda> llista_rondes:** Llista de les diferents rondes de la partida, van desde 0 fins num_rondes_maxim-1
- **private Sequencia sequencia_solucio:** Seqüència de colors intentada que simbolitza l'intent de resoldre el codi
- **private List<List<Integer>> solucio_maquina:** Solució que ha donat una màquina.

Mètodes principals

- **public boolean temps_excedit():** Retorna true si el temps usat > temps màxim de la partida segons la dificultat d'aquesta
- **public void codebreaker_guanya() & public void codemaker_guanya():** Aquestes funcions acaben la partida posant com a guanyador i perdedor els respectius codemaker i codebreaker
- **public void genera_sequencia_solucio_random(dificultats dif):** Funció la qual, a partir d'una dificultat, genera una solució random
- **public Double get_puntuacio():** Funció que serveix per calcular la puntuació obtinguda en una partida segons la dificultat d'aquesta i que servirà per fer l'assignació de punts als usuaris que hi han participat

2.2.8 Ronda

Aquesta classe fa referència a una ronda que es juga a una partida determinada. Per tant, l'identificador d'aquesta ronda dins d'una partida concreta és el seu num_ronda.

Conseqüentment, cada ronda identificada per un num_ronda és única en la partida de la qual forma part.

En aquesta ronda, es "juga" amb la seqüència intentada enviada pel codebreaker i es comprova aquesta seqüència intentada amb una seqüència de verificació del codi fet pel codemaker.

Atributs

- **private int id_partida:** Identificador únic, dins de la partida, de la ronda.
- **private int num_ronda:** Número de la ronda dins de la partida
- **private Sequencia sequencia_intentada:** Seqüència de colors intentada pel codebreaker
- **private Sequencia sequencia_verificacio:** Seqüència de verificació que posa el codemaker per verificar la sequencia_intentada

Mètodes principals

- **public Sequencia get_seq_intentada:** Mètode que retorna la Seqüència Intentada.
- **public Sequencia get_seq_verificacio:** Mètode que retorna la Seqüència de Verificació.
- **public void set_intentada:** Modifica la sequenica_intentada de la classe per la classe Sequencia que passem per paràmetre.
- **public void set_verificacio:** Modifica la sequenica_verificacio de la classe per la classe Sequencia que passem per paràmetre.
- **public boolean check_sequencia_encertada:** Comprova que la sequencia_verificacio sigui tot negre i envia

2.2.9 Seqüència

Aquesta classe fa referència a la seqüència de 4 colors, independentment de la dificultat de la partida, (tan intentada, per part del codebreaker, com de verificació, per part del codemaker).

Atributs

- **private colors[] array:** Array amb les 4 posicions de les boles. Les boles poden ser de diferents colors. Les seqüències intentades poden tindre fins al color num_color-1, mentre que la seqüència de verificació pot tindre els colors blanc, negre o no tindre bola(null)

Mètodes principals

- **public void set_position(int i, colors col):** Inserta un color en un posició determinada.
- **private List<Integer> toListInteger:** Retorna el array de colors en forma de llista de Integers.

2.2.10 Seqüència_intentada

Subclasse de seqüència que fa referència a l'intent de seqüència realitzat en una ronda d'una partida concreta.

En aquesta classe es comprova si la seqüència intentada pel codebreaker és correcta en termes de correcte posicionament dels colors i sense llocs buits.

Atributs

- **Tots els heredats de la superclasse Seqüència**

Mètodes principals

- **public void set_array(colors[] array, int num_colors):** Modifica el array de la classe per el que passem per paràmetre fent les comprovacions necessàries.

2.2.11 Seqüència_verificació

Subclasse de seqüència que fa referència a la seqüència de verificació realitzat en una ronda d'una partida concreta.

Es fa el posicionament d'espigues (tan blanques com negres) per fer la comprovació de la correctesa de la seqüència. Per exemple, a *valida_seq()* :: *bool* es comprova que estigui ben escrita (no de valor sinó de colors).

Atributs

- **Tots els heredats de la superclasse Seqüència**

Mètodes principals

- **public void set_array_verificacio(colors[] verificacio, colors[] solucio, colors[] intentada):** Modifica el array de la classe per el que passem per paràmetre fent les comprovacions necessàries
- **public pair<integer,integer> get_verificacio(colors[] arr_solucio, colors[] array):** Retorna un pair amb el nombre de espigues blanques i negres que tendria que retornar segons la solució y la intentada.
- **private static int get_pins_negres(colors[] arr_solucio, colors[] array_provat) && private static int get_pins_blanques(colors[] arr_solucio, colors[]**

array_provat): Funcio per calcular els pins negres i blanques donada una solució i un intent

- **private Pair<Integer, Integer> get_negres_blanques(colors[]**

verificacio_entrada): Funcio que retorna el nombre de pins negres i blanques que hi ha en una seqüència

2.2.12 Rànkking

Aquesta classe fa referència al rànkking que se li mostrarà al usuari. Aquest rànkking, que pot ser de tipus fàcil, normal, difícil o pvp, fent referència a al tipologia de partida que s'ha triat per jugar a l'inici, mostrarà un rànkking amb una classificació típica dels estàndards: apareixent l'usuari (amb el seu respectiu posicionament, i els seus punts en aquell tipus de rànkking).

D'aquesta manera, es pot seguir de manera fàcil i àgil l'evolució dels usuaris dins del joc, dins de cada dificultat i es poden realitzar comparacions a plena vista del desenvolupament de cada usuari (incloent-hi *user_maquina* que hi participen, menys al PvsP).

Atributs

- **protected List<Pair<Pair<Double, String>, LocalDate>> rank**: Llista que conté els usuaris amb els seus respectius punts i la data d'actualització d'aquesta posició del rànkking.

Mètodes principals

- **nova_partida_ranking(double punts_usuari, String nom_user)**: Es comprova a quina posició entra l'usuari. S'inserta els punts de l'usuari(a canviar a winrate) juntament amb el nom usuari i la data on s'ha entrat al ranking.

2.2.13 Record

Aquesta classe fa referència a un record en concret que forma part del sistema del joc. Aquest record és complert per un i només un usuari, el qual es pot anar transformant segons les variacions i actualitzacions de punts de part d'aquests usuaris del sistema.

Atributs

- **private String nom_record**: Nom del record que l'identifica (i s'avalua)
- **private String nom_usuari**: Nom de l'usuari que ha batut el récord anterior a el seu

- **private int modalitat_record:** Dificultat de la qual és el record.

Mètodes principals

- **La majoria de mètodes d'aquesta classe són d'utilitat simple i/o s'implementen a les subclasses**
-
- **check_if_record(int punts, String nom_usuari):** funció on es passen els punts que s'han fet en una partida i el nom de l'usuari que els ha aconseguit. Es comprova si supera al actual record i si el supera s'actualitza amb els paràmetres. Si no es supera es manté l'antic record.

2.2.14 RecordDouble

Aquesta subclasse de Record simbolitza el record de tipus puntuació.

Atributs

- **Tots els heredats de la superclasse Record**
- **private Double punts_record:** Punts del record en qüestió

Mètodes principals

- **public boolean actualitza(Object valor, String nom_user):** Funció per comprovar si es bat el record actual o no (en aquest cas, record de puntuació segons la dificultat indicada a la inicialització del record)

2.2.15 RecordInteger

Aquesta subclasse de Record simbolitza el record de tipus ratxa (o streak).

Atributs

- **Tots els heredats de la superclasse Record**
- **private Integer streak_record:** Ratxa de victòries del record en qüestió

Mètodes principals

- **public boolean actualitza(Object valor, String nom_user):** Funció per comprovar si es bat el record actual o no (en aquest cas, record de ratxa segons la dificultat indicada a la inicialització del record)

2.2.16 RecordLong

Aquesta subclasse de Record simbolitza el record de tipus temps.

Atributs

- **Tots els heredats de la superclasse Record**
- **private Long temps_record:** Temps del record en qüestió

Mètodes principals

- **public boolean actualitza(Object valor, String nom_user):** Funció per comprovar si es bat el record actual o no (en aquest cas, record de temps segons la dificultat indicada a la inicialització del record)

2.2.17 Type_user

Enumeration que permet identificar la tipologia de l'usuari que fa referència a la classe User. D'aquesta manera, cada usuari serà d'un tipus o de l'altre i això ens permetrà desenvolupar diferents funcionalitats i mostrar errors en funció del tipus d'usuari que sigui.

Aquests tipus són:

1. user_persona,
2. user_maquina

2.2.18 dificultats

Enumeration que identifica el tipus de dificultat del qual és una partida en concret.

Aquets enumeration consta de diferents atributs (que es mostren a continuació) que prenen valors diversos en funció del tipus de dificultat que es tracti.

1. private String dificultat;
2. private int num_dificultat;
3. private int num_colors;
4. private int temps_max;
5. private int num_max_rondes;

- Si es tracta d'una dificultat fàcil, aquests atributs prenen els valors de:
FACIL("facil",1,4,900L,14)
- Si es tracta d'una dificultat normal, aquests atributs prenen els valors de:
NORMAL("normal",2,6,720L,10)
- Si es tracta d'una dificultat difícil, aquests atributs prenen els valors de:
DIFICIL("difcil",3,8,600L,6)

D'aquesta manera, ens permet distingir el tipus de partida a jugar, la seva dificultat, la seva jugabilitat i tot el que implica per tema rànkig,...

2.2.19 colors

Enumeration que ens permet classificar els colors (assignant-los un id únic per a cada un). D'aquesta manera, podríem crear seqüències d'intent i de verificació més còmodament i seria una metodologia d'identificació de les mateixes ràpida, fàcil de modificar i amb una alta usabilitat.

Aquests colors són i estan associats a:

1. NULL("null",0),
2. VERMELL("vermell",1),
3. VERD("verd",2),
4. BLAU("blau",3),
5. GROC("groc",4),
6. MAGENTA("magenta",5),
7. CIAN("cian",6),
8. MARRO("marro",7),
9. GRIS("gris",8),
10. BLANC("blanc",9),
11. NEGRE("negre",10);

2.2.20 Palr

Aquesta classe fa referència al @override que fem a la classe ja implementada Pair.

Aquestes substitucions s'utilitzaran en molts dels atributs de les classes comentades anteriorment, com per exemple al rànkig.

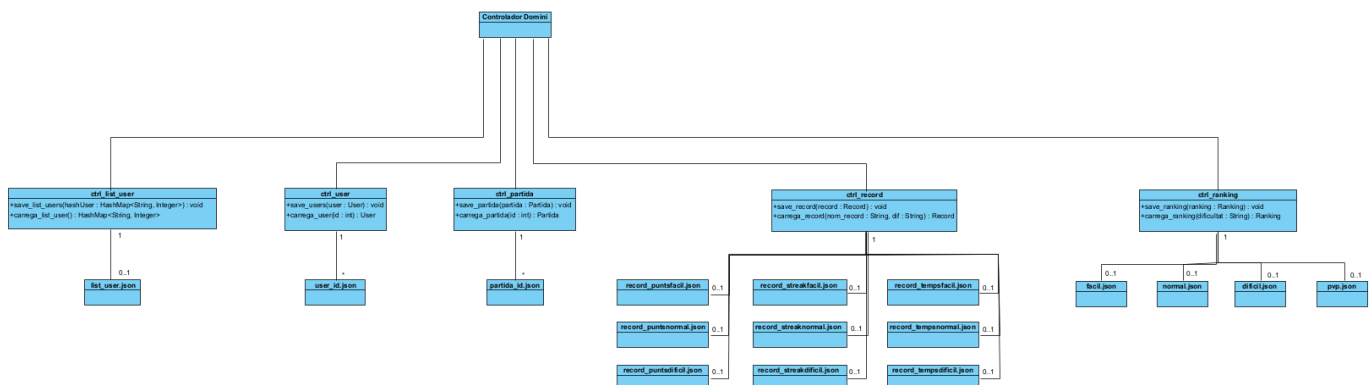
Naturalment, els atributs d'aquesta classe són:

1. private final A first;
2. private final B second;

3. Capa de persistencia

3.1 Diagrama de la capa de persistència

Aquí proposem una imatge del nostre diagrama de classes de la capa de presentació. Per a veure el diagrama millor i en més detall, dirigir-se a la carpeta de DOCS i seleccionar el document pertinent.



3.2 Descripció de les classes

La nostra capa es basa en diferents controladors, un per a cada tipus de dades que volem guardar. Guardarem tota la informació en el path: "/EXE/dades", i aquesta informació es guardarà en format Json, ja que farem servir la llibreria "Gson-2.8.5.jar" de Google.

Diferents controladors:

- ctrl_list_user
- ctrl_user
- ctrl_partida
- ctrl_ranking
- ctrl_record

3.2.1 ctrl_list_user

El “ctrl_list_user” s’encarrega de guardar/carregar les dades del HashMap<String,Integer> de la capa de domini. El path on es guardarà el Json de la llista serà: “/EXE/dades/list_user.json”.

- Mètodes:

- *void save_list_users(HashMap<String,Integer> hashuser):*

Aquest mètode s’encarrega de crear el fitxer “.json” i el parth de directoris on estarà guardat, en aquest cas “/EXE/dades”, en cas de que no existeixin. Després de crear el fitxer on es guardarà, escrivim el HashMap en format json en el fitxer “list_user.json”.

- *HashMap<String,Integer> carrega_list_user():*

Aquest mètode s’encarrega de retornar el “HashMap<String,Integer>” que es llegeix del fitxer “list_user.json”, sempre i quan el fitxer existeix, en cas contrari retorna null.

3.2.2 ctrl_user

El “ctrl_user” s’encarrega de guardar/carregar les dades d’una instància de la classe “User” de la capa de domini. El path on es guardarà el Json de la llista serà: “/EXE/dades/users/”id del usuari”.json”.

- Mètodes:

- *void save_users(User user):*

Aquest mètode s’encarrega de crear el fitxer “.json” i el parth de directoris on estarà guardat, en aquest cas “/EXE/dades/users/”, en cas de que no existeixin. Després de crear el fitxer on es guardarà, escrivim el l’objecte “user” en format json en el fitxer “id del usuari”.json”, sent el “id del usuari l’identificador únic d’aquest.

- *User carrega_user(int id):*

Aquest mètode s’encarrega de retornar una instància de la classe “User”, amb el identificador únic “id” que es llegeix del directori “/EXE/dades/users/”, sempre i quan el fitxer existeix, en cas contrari retorna null.

3.2.3 ctrl_partida

El “ctrl_partida” s’encarrega de guardar/carregar les dades de una instància de la classe “Partida” de la capa de domini. El path on es guardarà el Json de la llista serà: “/EXE/dades/partides/’id de la partida’.json”.

- Mètodes:

- *void save_partida(Partida partida):*

Aquest mètode s’encarrega de de crear el fitxer “.json” i el parth de directoris on estarà guardat, en aquest cas “/EXE/dades/partides/”, en cas de que no existeixin. Després de crear el fitxer on es guardarà, escrivim el l’objecte “partida” en format json en el fitxer “id de la partida”.json”, sent el “id de la partida” el identificador únic d’aquest.

- *Partida carrega_partida(int id):*

Aquest mètode s’encarrega de retornar una instància de la classe “Partida”, amb el identificador únic “id” que es llegeix del directori “/EXE/dades/partides/”, sempre i quan el fitxer existeix, en cas contrari retorna null.

3.2.4 ctrl_ranking

El “ctrl_ranking” s’encarrega de guardar/carregar les dades de una instància de la classe “Ranking” de la capa de domini. El path on es guardarà el Json de la llista serà: “/EXE/dades/rankings/’dificultat’.json”.

- Mètodes:

- *void save_ranking(Ranking ranking):*

Aquest mètode s’encarrega de de crear el fitxer “.json” i el parth de directoris on estarà guardat, en aquest cas “/EXE/dades/rankings/”, en cas de que no existeixin. Després de crear el fitxer on es guardarà, escrivim el l’objecte “ranking” en format json en el fitxer “dificultat”.json”, sent el “dificultat” el identificador únic de la dificultat del ranking, com per exemple seria “facil.json”

- *Ranking carrega_ranking(String dificultat):*

Aquest mètode s’encarrega de retornar una instància de la classe “Ranking”, amb el identificador únic “dificultat” que es llegeix del directori

“/EXE/dades/rankings/”, sempre i quan el fitxer existeix, en cas contrari retorna null.

3.2.5 ctrl_record

El “Ctrl_record” s’encarrega de guardar/carregar les dades de una instància de la classe “Record” de la capa de domini. El path on es guardarà el Json de la llista serà: “/EXE/dades/records/nom_record + modalitat record”.json”.

- Mètodes:

- *void save_record(Record record):*

Aquest mètode s’encarrega de crear el fitxer “.json” i el parth de directoris on estarà guardat, en aquest cas “/EXE/dades/records/”, en cas de que no existeixin. Després de crear el fitxer on es guardarà, escrivim el l’objecte “record” en format json en el fitxer “nom_record + modalitat record”.json”, sent el “nom_record” el identificador únic del nom del record, i “modalitat_record” seria el tipus de dificultat en que s’ha fet el record, un exemple seria “puntsdifícil.json”

- *Record carrega_record(String nom_record, String dif):*

Aquest mètode s’encarrega de retornar una instància de la classe “Record”, amb els identificadors únics “nom_record” i “dif” que es llegeix del directori “/EXE/dades/records/”, sempre i quan el fitxer existeix, en cas contrari retorna null.

4. Estructura de dades i algorismes

4.1 Estructures de dades

En aquesta secció es comenten les estructures de dades que hem utilitzat i perquè ho hem fet.

En aquesta segona entrega, i com que està molt relacionada amb la tercera, les estructures de dades utilitzades han variat respecte a les de la primera, ja que amb la possibilitat de l'ús de la persistència i dades, hem pogut prescindir d'algunes estructures.

Per una banda, en el controlador domini hem pogut prescindir dels `hashMaps` on guardàvem els usuaris, records i rankings per fer totes les modificacions. Tot i que amb aquesta estructura de dades ens donava un cost mitjà constant de temps en les 3 operacions, fent ús de l'arquitectura de tres capes amb la persistència, és molt millor.

Per altre banda, al controlador partida hem eliminat el `HashMap` on guardàvem les partides que es jugaven, ja que, igual que com fem amb els usuaris, records i rankings, utilitzem la persistència per guardar les partides.

Hem seguit fent servir un array de l'enumeration *colors* per guardar les seqüències de "boletes" que s'entren, ja sigui per provar d'encertar com de verificar si l'anteriorment mencionada és correcte o no.

Dins de la classe `User_persona` (abans a `User`) hem seguit fent servir l'estructura `ArrayList` per guardar les partides però, aquesta vegada, només hem guardat el `integer` identificador que les identificava per així no haver de guardar tot l'objecte partida dins de l'estructura i haver-la d'anar actualitzant constantment. D'aquesta manera, amb una simple consulta al `id` pertinent, podem consultar la partida guardada gràcies a la persistència aplicada.

Dins de `Ranking` hem seguit guardant una `LinkedList` de dos pairs amb el `double ratio`, el `string` del nom del usuari situat el `rànk` i la data de finalització de la partida amb una simple `LocalDate` que ens permet tenir constància de quan s'ha actualitzat el seu valor.

Això ens podria ser útil per mostrar el `rànk` d'usuaris per pantalla i facilitaria el reconeixement de cada un dels usuaris degut a la gran comoditat que resulta poder veure ordenats per punts els participants del joc (i inclús fa quant que no juguen una partida d'aquella dificultat, és a dir, fa quan no se'ls hi ha actualitzat el posicionament al `rànk` global d'aquella dificultat en específic).

Finalment, i parlant de les dificultats, hem utilitzat un enumeration per definir les nostres pròpies, diferenciant-les pel número de colors a endevinar, el temps màxim de partida i el número màxim de ronda per a completar-la. Això ho hem fet amb un enumeration que es redefineix amb atributs i mètodes.

4.2 Algoritmes

Five Guess Algorithm

L'algorisme Five Guess és una estratègia per resoldre el joc de Mastermind que consisteix a fer un nombre limitat d'intents basats en una selecció cuidadosa dels codis proposats, amb la finalitat de reduir ràpidament l'espai de solucions possibles fins a trobar la solució correcta. Primer de tot una descripció general de què fa l'algorisme:

1. El mètode "solve" rep una solució i retorna una llista d'intents per resoldre el joc.
2. Es inicialitza una llista buida d'intents anomenada "guesses".
3. S'obtenen els colors possibles cridant la funció "get_colors()".
4. Es generen tots els codis possibles cridant la funció "**generateCodes()**" i s'emmagatzemen a la llista "possibleCodes".
5. Es defineix un intent inicial recomanat per l'algorisme, [1, 1, 2, 2]. Aquest intent s'afegeix a la llista d'intents ("guesses").
6. Es calcula el resultat de l'intent comparant-lo amb la solució utilitzant la funció "getScore()". El resultat s'emmagatzema a la llista "result".
7. Mentre l'intent no sigui igual a la solució, es realitza el següent:
 - a. Es filtren els codis possibles en base a l'intent i al resultat utilitzant la funció "**filterCodes()**". Els codis que tinguin el mateix resultat que l'intent s'afegeixen a la llista "filteredCodes".
 - b. S'obté el millor intent possible cridant la funció "**getBestGuess()**" passant com a argument la llista "filteredCodes".
 - c. Es calcula el resultat del nou intent i s'actualitza la llista "result".
 - d. El nou intent s'afegeix a la llista d'intents ("guesses").
8. Un cop es troba la solució, es retorna la llista d'intents.

A continuació, es descriuen les funcions i estructures de dades utilitzades en l'algorisme:

List

És una estructura de dades que s'utilitza per emmagatzemar una seqüència d'elements ordenats en una llista. En aquest algorisme, s'utilitza una llista d'enters per representar les seqüències de colors del codi a endevinar i dels codis proposats.

Arrays

És una classe d'utilitat a Java que proporciona diversos mètodes útils per treballar amb matrius (arrays). En aquest algorisme, s'utilitza el mètode `asList()` de la classe `Arrays` per crear una llista d'enters a partir d'un array d'enters.

ArrayList

És una implementació de la interfície `List` que utilitza una matriu dinàmica per emmagatzemar elements. En aquest algorisme, s'utilitza un `ArrayList` per emmagatzemar les seqüències de colors de tots els codis possibles i dels codis proposats.

Math.pow()

És un mètode estàtic de la classe `Math` a Java que retorna la potència d'un nombre donat. En aquest algorisme, s'utilitza `Math.pow()` per calcular el nombre de codis possibles que poden ser generats a partir d'una llista de colors i una longitud donada.

Random

És una classe a Java que s'utilitza per generar nombres aleatoris. En aquest algorisme, no s'utilitza explícitament la classe `Random`, però la selecció dels codis proposats es basa en un algorisme de selecció aleatòria.

generateCodes()

És una funció que genera tots els codis possibles a partir d'una llista de colors i una longitud donada. Utilitza un algorisme d'enumeració exhaustiva que genera tots els codis possibles utilitzant row nested loops (el vem aprendre a DBD). També podria haver utilitzat una funció recursiva. En termes de eficiència els 2 algoritmes (row nested i recursiu) son iguals $O(C^n)$ on **C** es el nombre de colors i **n** el nombre de ranures (en el nostre cas 4). No obstant vem llegir que el row nested és més eficient ja que evita la sobrecarga recursiva i la creació d'una gran quantitat d'objectes en el procés de generació.

getScore()

És una funció que retorna el resultat (blanques i negres) d'una possible solució. Prendre com a entrada una seqüència de colors (intent) i la seqüència de colors correcta (solució), i retorna una llista d'enters que representa el nombre de fitxes negres i blanques obtingudes en l'intent. Aquesta crea una instància de seqüència verificació i crida la funció `get_verificacio()`.

getColors()

`getColors` es una funció que retorna un array de integers del 1 al nombre de colors que indica la dificultat de la partida: agafa la dificultat que se li assigna i crida el mètode `get_num_colors()`. Després omple un array amb els integers corresponents.

filterCodes()

La funció `filterCodes` s'encarrega de filtrar els codis possibles en base a un intent i el seu resultat. Prendrà com a entrada la llista de tots els codis possibles (`possibleCodes`), l'intent actual (`guess`) i el resultat d'aquest intent (`result`).

L'objectiu d'aquesta funció és reduir la llista de codis possibles descartant aquells que no podrien ser la solució en funció del resultat obtingut. Per fer-ho, iterarà sobre cada codi de la llista `possibleCodes` i compararà el resultat d'aquest codi amb el resultat de l'intent actual (`guess`). Si el resultat és el mateix, s'afegirà aquest codi a la llista de codis filtrats (`filteredCodes`).

Aquesta funció és essencial en l'algoritme "Five Guess" ja que permet reduir l'espai de cerca de les possibles solucions en cada iteració. En filtrar els codis possibles, s'eliminen aquells que no podrien ser la solució en funció de la informació proporcionada pel resultat de l'intent actual.

getBestGuess()

La funció `getBestGuess` s'encarrega de determinar el millor intent possible a partir d'una llista de codis possibles. Prendrà com a entrada la llista de codis possibles (`possibleCodes`).

L'objectiu d'aquesta funció és trobar l'intent que, en funció dels codis restants possibles, maximitzi la reducció de l'espai de cerca en el següent pas. Per aconseguir-ho, la funció iterarà sobre cada codi de la llista `possibleCodes` i calcularà el nombre màxim de

possibilitats restants després d'aquell intent. Llavors, seleccionarà l'intent que minimitzi aquest nombre màxim de possibilitats restants.

Aquesta funció és clau en l'algoritme "Five Guess" ja que permet seleccionar el millor intent en cada iteració, basant-se en l'objectiu de maximitzar la reducció de l'espai de cerca. La selecció del millor intent es basa en l'avaluació de tots els intents possibles i la seva influència en la reducció del nombre total de possibilitats.

Per el que respecta a la performance del algoritme five guess hem obtinguts millors resultats que la primera entrega. Amb l'ajuda de un driver que executa la interfície solve de forma massiva hem pogut provar i comparar el dos algoritmes. El driver genera un nombre donat pel usuari de codis randoms i executa la interfície solve per a cada un d'aquest, enregistra amb quantes rondes resol l'algoritme el codi i si realment l'ha resolt correctament. Vem fer una prova amb 100 codis i aquest va ser el resultat:

Dificultat	núm Rondas
Baja	3.356
Media	4.56
Alta	5.93

Com podem veure els resultats són bastant bons i baixos (serà complicat que el genetic el guanyi)

Genetic Algorithm

Un algorisme genètic és un algorisme metaheuristic que es basa en la selecció natural. Utilitza operacions biològiques com ara l'encreuament de dos pares per a generar un fill, la el qual pot mutar o permutar per donar un component de canvi aleatori. El seu comportament es pot definir amb els passos següents:

1. Posem iteració = 1
2. Provem un intent fix(es recomana el {1,1,2,3} a diferència del Five-Guess)
3. Obtenim els pins blancs/negres d'aquest intent inicial.
4. Mentre num_negres != tamany_intents:
 - 4.1. incrementa la iteració
 - 4.2. Crea un conjunt de "elegibles" i posa la generació a 1
 - 4.3. Inicialitza la població de manera random, sense que hi hagin repeticions
 - 4.4. Mentre no superem la màxima generació i el conjunt de elegibles no sigui més gran del que pot ser:
 - 4.4.1. Genera una nova població a través de creuament, mutació, permutació i inversió.
 - 4.4.2. Calcula el "fitness" de cada individu
 - 4.4.3. afegeix les combinacions elegibles al conjunt de elegibles(si no hi són ja)
 - 4.4.4. incrementa la generació
 - 4.5. Juga un intent dels elegibles
 - 4.6. Obté pins blancs/negres corresponents al intent
5. Final

Elegibles: El conjunt de elegibles són tots aquells que, per a cada intent jugat prèviament, o bé tenen el mateix nombre de pins blancs, o bé tenen el mateix nombre de pins negres. Això es fa per determinar quins intents són interessants de creuar per a generar la solució amb més probabilitat. Per a calcular la probabilitat de que un codi sigui elegit com a pare d'un nou intent, determinem el "fitness".

Fitness: El fitness es la mesura que "quantifica" com de bo es un intent. Això es fa tenint em compte la diferència entre pins blanc i la diferència entre pins negres. El fitness d'un intent l'hem calculat de la següent manera:

$$f(c; i) = a \left(\sum_{q=1}^i |negres_anterior_q - negres_intent_q| \right) + \sum_{q=1}^i |blanques_anterior_q - blanques_intent_q| + bP(i - 1)$$

negres_anterior/blanques_anterior: pins en cas que l'anterior intent fos la solució.

negres_intent/blanques_intent: Pins que ha obtingut l'intent anterior respecte la resposta.

a: pes dels pins negres respecte pins blancs(el millor valor és 1)¹.

b: pes de la constant per a calcular la importancia relativa de l'intent comparat amb el número de pins blancs i negres(el millor valor és 2)¹.

Selecció de “pares” per generar noves poblacions: Per a decidir quins cromosomes generaran els components de la següent generació farem servir un algorisme de ruleta. Aquest funciona de la següent manera: agafem un valor random entre 0 i 1. Anem recorrent els diferents cromosomes elegibles, i quan la suma dels fitness normalitzats dels cromosomes supera el valor random, agafem el cromosoma que hi ha en aquella posició. És un algorisme senzill, però afavoreix cromosomes amb alts fitness, ja que aquests abarquen un rang més gran. També hi ha altres algorismes com la selecció elitista, selecció mitjançant torneigs, selecció per ranking, etc.

Estructures de dades i funcions utilitzades:

- **List:** És l'estructura amb la que més treballem, ja que per als intents farem servir aquesta estructura. Potser no és la millor, ja que el tamany dels intents és fix en el nostre cas, però per a guardar els conjunts de Cromosomes i de solucions és ideal ja que el tamany va variant.
- **Cromosoma:** Se'n parlarà més endavant d'aquesta classe. Representa un intent, i té les operacions de mutació, permutació, encreuament singular i doble i obtenció de pins en comparació amb un paràmetre que se li passa.
- **Pair:** No es fa servir massa, però es fa servir. Aquesta estructura la fem servir sobretot per retornar els valors de pins blancs i negres.
- **init_poblacio():** Genera una població de POPULATION_SIZE random i sense repeticions.
- **Genera_nova_poblacio(List<Integer> codi_sol, List<Cromosoma> codis_previs):** Genera una nova població a través de mutacions, permutacions i encreuaments. Els encreuaments poden ser d'un sol punt o de doble amb una probabilitat de CROSSOVER_RATE. Encreuament d'un sol punt o de dos punts es diferencia en els intervals en que s'agafa el valor d'un pare o de l'altre.
- **Barreja_poblacio(List<Cromosoma> poblacio):** Barreja els cromosomes de la població.

¹ Berghman, L., Goossens, D., & Leus, R. (2009). Efficient solutions for Mastermind using genetic algorithms. *Computers & Operations Research*, 36(6), 1880–1885. <https://doi.org/10.1016/j.cor.2008.06.004>

- **Muta_poblacio(List<Cromosoma> poblacio):** Muta els cromosomes de la població amb una probabilitat de MUTATION_RATE
- **Permuta_poblacio(List<Cromosoma> poblacio):** Permuta els cromosomes de la població amb una probabilitat de PERMUTATION_RATE.
- **fitness(Cromosoma c, List<Cromosoma> cromosomes_previs, List<Integer> codi_solucio):** Calcula el fitness d'un cromosoma segons la formula comentada prèviament
- **exists(Cromosoma c, List<Cromosoma> poblacio):** Comprova si un cromosoma existeix en una població.

Comparació de la performance d'algoritmes

Dificultat	mitja de nombre de rondes	
	Five Guess	Genetic
Baja	3.066	2.95
Media	4.08	4.26
Alta	5.61	5.28

Els resultats anteriors son per a 100 intents amb les mateixes seqüències solució per els dos algoritmes. Veiem com tenen una performance molt semblant. Tot i que en teoria el algoritme genetic hauria de ser millor que el five guess com més colors son possibles veiem que també guanya el genetic quan hi han 4 colors a la dificultat baixa tot i que són molt aprop i si tinguéssim en compte les desviacions típiques veuríem que no podem determinar quin dels dos és millor, ja que l'error seria suficientment gran com abarcar les dues possibilitats. En el cas de la dificultat alta veiem com si que hi ha més diferència i per tant podem dir que quan més colors millor el genetic es transforma comparant-lo amb el five guess.